BIBLIOWEB

Endpoints faltantes para la interoperabilidad con el backend

Documento técnico de integración Frontend - API REST .NET
Fecha: 31 de octubre de 2026

Resumen ejecutivo

Este documento identifica los endpoints que el backend debe exponer para que el frontend de BiblioWeb pueda operar con datos reales y dejar de depender de datos simulados, arreglos estáticos y localStorage.
El análisis se realizó sobre la estructura actual del frontend y su README. Actualmente solo se consume de forma efectiva el inicio de sesión; las funcionalidades de libros, usuarios, préstamos y dashboard todavía no están conectadas a la API.

Estado actual de la integración

Endpoint actualmente utilizado:
POST /api/auth/login
La respuesta esperada puede ser el usuario directamente o estar contenida en la propiedad user. Debe incluir name, email, role y active.
Rutas declaradas pero aún no conectadas:
GET /api/dashboard GET /api/books GET /api/users GET /api/loans

Prioridad de implementación

Prioridad Alta: necesaria para reemplazar datos locales y habilitar los flujos principales.
Prioridad Media: necesaria para completar requisitos del sistema y la administración.
Prioridad Técnica: necesaria para operación, seguridad y mantenimiento de la integración.

1. Autenticación y usuarios

Alta | POST /api/auth/register | Registrar lectores o estudiantes
Alta | GET /api/auth/me | Obtener el usuario autenticado y validar la sesión
Alta | POST /api/auth/logout | Cerrar la sesión en el backend
Media | POST /api/auth/refresh | Renovar el token, si se utiliza JWT con refresh token
Media | POST /api/auth/forgot-password | Solicitar recuperación de contraseña
Media | POST /api/auth/reset-password | Restablecer contraseña
Alta | GET /api/users | Listar usuarios para administración
Alta | GET /api/users/{id} | Consultar detalle de un usuario
Alta | POST /api/users | Crear un usuario
Alta | PUT/PATCH /api/users/{id} | Editar datos, rol o estado
Alta | PATCH /api/users/{id}/status | Activar o desactivar un usuario
Alta | DELETE /api/users/{id} | Eliminar o retirar un usuario

2. Libros y catálogo

Alta | GET /api/books | Listar libros
Alta | GET /api/books/{id} | Consultar un libro
Alta | POST /api/books | Registrar un libro
Alta | PUT/PATCH /api/books/{id} | Editar información del libro
Alta | DELETE /api/books/{id} | Eliminar o desactivar un libro
Alta | GET /api/books/search | Buscar por título, autor, categoría o ISBN
Alta | GET /api/books/{id}/availability | Consultar disponibilidad actual
Media | GET /api/categories | Obtener categorías disponibles
El modelo debe contemplar estados como Disponible, Reservado, En préstamo y En mantenimiento, además de ISBN, ejemplares totales y ejemplares disponibles.

3. Ejemplares físicos

Alta | GET /api/books/{bookId}/copies | Listar ejemplares de un libro
Alta | POST /api/books/{bookId}/copies | Registrar un ejemplar
Alta | GET /api/copies/{id} | Consultar un ejemplar
Alta | PATCH /api/copies/{id}/status | Cambiar estado del ejemplar
Alta | DELETE /api/copies/{id} | Retirar un ejemplar
La disponibilidad debe controlarse por ejemplar físico, no solamente por título.

4. Préstamos

Alta | GET /api/loans | Listar préstamos
Alta | GET /api/loans/{id} | Consultar un préstamo
Alta | GET /api/users/{userId}/loans | Consultar préstamos de un lector
Alta | POST /api/loans | Registrar un préstamo
Alta | PATCH /api/loans/{id}/return | Registrar devolución
Alta | PATCH /api/loans/{id}/renew | Renovar el plazo
Media | PATCH /api/loans/{id}/cancel | Cancelar un préstamo
Media | GET /api/loans/overdue | Listar préstamos vencidos

Solicitud sugerida para crear un préstamo:
{"userId": 2, "copyId": 15, "dueDate": "2026-11-15"}
El backend debe impedir prestar ejemplares no disponibles, usuarios inactivos o bloqueados, y exceder los límites configurados.

5. Reservas

Alta | GET /api/reservations | Listar reservas para administración
Alta | GET /api/users/{userId}/reservations | Consultar reservas del lector
Alta | GET /api/reservations/{id} | Consultar una reserva
Alta | POST /api/reservations | Solicitar una reserva
Alta | PATCH /api/reservations/{id}/approve | Aprobar una reserva
Alta | PATCH /api/reservations/{id}/reject | Rechazar una reserva
Alta | PATCH /api/reservations/{id}/cancel | Cancelar una reserva
Alta | PATCH /api/reservations/{id}/complete | Marcarla como retirada
Media | GET /api/reservations/pending | Listar reservas pendientes
Estados recomendados: Pendiente, Aprobada, Rechazada, Cancelada, Retirada y Expirada.

6. Dashboard

Alta | GET /api/dashboard/summary | Indicadores generales
Media | GET /api/dashboard/recent-activity | Actividad reciente
Media | GET /api/dashboard/alerts | Alertas
Media | GET /api/dashboard/loans-by-status | Préstamos agrupados por estado
Media | GET /api/dashboard/books-availability | Disponibilidad de libros
Media | GET /api/dashboard/overdue | Resumen de retrasos

Respuesta sugerida:
{"activeLoans":184, "availableBooks":1240, "activeUsers":426, "overdueLoans":17, "recentActivity":[], "alerts":[]}

7. Sanciones y multas

Alta | GET /api/fines | Listar multas
Alta | GET /api/users/{userId}/fines | Consultar multas de un usuario
Alta | GET /api/fines/{id} | Consultar una multa
Media | POST /api/fines | Registrar una sanción manual
Alta | PATCH /api/fines/{id}/pay | Registrar pago o regularización
Media | PATCH /api/fines/{id}/waive | Anular una multa autorizadamente
Alta | GET /api/users/{userId}/account-status | Consultar bloqueo o estado de cuenta

8. Endpoints técnicos y contrato

GET /api/health | Verificar disponibilidad de la API
GET /swagger/v1/swagger.json | Consultar el contrato OpenAPI
GET /api/metadata | Obtener estados, roles y catálogos permitidos

Contrato mínimo recomendado

Todos los endpoints protegidos deben documentar autenticación, roles permitidos, parámetros, DTO de entrada, DTO de salida, códigos HTTP y errores de validación.
Códigos HTTP esperados:
200/201: operación exitosa; 204: operación exitosa sin contenido; 400: datos inválidos; 401: no autenticado; 403: sin permisos; 404: recurso inexistente; 409: conflicto de negocio; 422: regla de negocio no cumplida; 500: error interno.

Formato de error sugerido:
{"status": 409, "code": "BOOK_NOT_AVAILABLE", "message": "El ejemplar no está disponible.", "details": []}

Recomendaciones de integración

• Alinear los tipos TypeScript con los DTOs reales del backend.
• Reemplazar localStorage y arreglos estáticos por servicios HTTP por feature.
• Centralizar autenticación, encabezados, manejo de errores y expiración de sesión en el cliente API.
• Usar TanStack Query para caché, invalidación y estados de carga.
• Aplicar las reglas de negocio en el backend; el frontend solo debe anticipar y reflejar errores.
• Proteger las operaciones administrativas con autorización por rol.

• Publicar Swagger/OpenAPI como fuente única de verdad para el contrato.

Conclusión

El mínimo necesario para conectar el frontend con datos reales comprende autenticación, usuarios, libros, préstamos, reservas, multas y dashboard. El equipo debe confirmar con backend los nombres definitivos, DTOs, estados y mecanismo de autenticación antes de implementar los servicios en el frontend.